

Embedding Inferred JML Contracts in Java Bytecode and OpenAPI Specifications

Anonymous Authors

Affiliation withheld for double-anonymous review

Abstract—Consumers of compiled Java libraries and remote REST services rarely have the source code, which makes automatically inferred Java Modeling Language (JML) specifications unavailable to the consumers that would benefit from them: test generators, IDEs, verifiers, and large language models reading APIs at runtime. We propose a uniform mechanism for distributing inferred JML contracts: a runtime-retained annotation, `@JmlSpecs`, written into class-file attributes by an ASM-based embedder, plus a parallel OpenAPI 3.x extension family (`x-jml-*`) for service boundaries. We evaluate the embedder on six real-world Java libraries (Commons Lang, IO, and Math, Guava, jOOL, and Vavr; 39,268 methods) with synthetic specifications and on a real-inferred-specs corpus of 22,881 specs across the same six libraries’ source trees. The OpenAPI extension is exercised against the reference parser `swagger-parser 2.1.x`. We measure 100.0% lossless roundtrip across all thirteen library-regime runs; 4.80–6.97% bytecode-size overhead on synthetic specs and 4.63–10.20% on real inferred specs; embedding throughput 6,000–34,000 methods/second and reading throughput 68,000–334,000 methods/second. A format-optimisation step (single packed annotation per method, default-omission, and a twelve-token dictionary) reduces byte overhead by 31% on real inference and 48–55% on synthetic specs over a per-clause baseline. A negative test confirms that embedded classes load correctly in JVMs that have no awareness of the annotation type. Any existing JAR can be enriched in place, with no build-toolchain changes.

Index Terms—JML, Java bytecode, ASM, OpenAPI, specification distribution, runtime annotations, software contracts

I. INTRODUCTION

Recent work has shown that automatically inferred Java Modeling Language (JML) specifications materially improve LLM-generated unit tests on real-world Java code [1]. That result, and others like it on inferred specifications for verification and tooling, share an unstated assumption: the consumer of the specifications — the test generator, the IDE, the verifier, the language model reading the API at inference time — has access to the same source tree the inferrer ran over. The assumption is rarely satisfied in practice. Java libraries are distributed as JAR files; REST endpoints expose only their HTTP signature; gRPC services expose only their protobuf descriptors. Source code is the exception, not the norm.

The conceptual question of whether specifications *should* be transported alongside compiled artefacts has a long history [2]–[4]. The practical question is whether the standard toolchain

can be made to carry them without modification. In this paper, we propose and evaluate a uniform mechanism for distributing inferred JML contracts in compiled Java libraries and in REST service interfaces: a runtime-retained annotation written into class-file attributes by an ASM-based embedder, plus a parallel OpenAPI 3.x extension family for service boundaries.

We address the following research questions:

- **RQ1.** Can inferred JML contracts be embedded in compiled Java artefacts and reproduced losslessly from the embedded bytes, with no source-code dependency?
- **RQ2.** What is the bytecode-size overhead of the embedding, and how does it scale across libraries of different shapes (utility code, I/O abstractions, generics-heavy collections)?
- **RQ3.** Which format-design choices most reduce that overhead — a single packed annotation per method, default-omission of the dominant clause, a learned token dictionary, per-spec compression — and what is the per-choice impact on real inferred specs versus synthetic specs?
- **RQ4.** Is the embedding and reading throughput sufficient to support per-pull-request continuous-integration workflows on production-scale libraries?
- **RQ5.** Does an OpenAPI extension family conform to OpenAPI 3.x’s extension rule, roundtrip losslessly through a reader/writer pair, and survive the canonical `swagger-parser` unchanged?

We evaluate the bytecode side empirically on six real-world Java libraries (Commons Lang, IO, and Math, Guava, jOOL, and Vavr; 39,268 methods) and the inferrer-to-embedder pipeline on real inferred specs across all six (22,881 specs). The OpenAPI side is established by construction against OpenAPI 3.x’s `x-*` extension rule and empirically by parsing a document carrying every `x-jml-*` key through `swagger-parser 2.1.x` and verifying that all five keys, value lists, and the `signals` clause’s nested objects are preserved. Table I summarises the findings: 100.0% lossless roundtrip on every library-regime run; 4.80–6.97% bytecode overhead under synthetic specs and 4.63–10.20% under real inferred specs; a format-optimisation step reduces overhead by 31% on real inference and 48–55% on synthetic; embedding throughput is 6,000–34,000 m/s, with reading throughput an order of magnitude higher.

Contributions. In summary, this paper makes the following

Replication package (embedder/extractor source, JSON Schema, measurement logs, reproduction scripts): <https://anonymous.4open.science/r/jml-inferrer-anon> during review; non-anonymous DOI-archived release in the camera-ready version.

contributions:

- A Java annotation type, `@JmlSpec`, repeatable per clause, written by an ASM-based embedder into class-file `RuntimeVisibleAnnotations` attributes, from which reflection or any class-file reader recovers the embedded specification.
- An OpenAPI 3.x extension family (`x-jml-requires`, `x-jml-ensures`, `x-jml-assignable`, `x-jml-signals`, `x-jml-invariant`) carried per-operation and per-schema, conformant with the OpenAPI 3.x `x-*` extension rule and roundtripped losslessly by the supplied reader/writer.
- A Maven classifier sidecar (`<artifact>-jml.jar`) carrying JML stub files, used when in-bytecode embedding is unsuitable (signed JARs that cannot be re-signed; specifications too large to fit the per-class budget).
- An empirical evaluation across 39,268 methods of six real-world Java libraries (synthetic specs) and 22,881 real inferred specs across the same six libraries’ source trees, showing a 100.0% lossless-roundtrip rate, 4.63–10.20% bytecode-size overhead, and embedding/reading throughput high enough to support per-pull-request continuous-integration workflows.
- A format-optimisation step (single packed annotation per method, default-omission for `assignable \nothing`, and a twelve-token dictionary for common JML constructs) that reduces byte overhead by 31% on real inference and 48–55% on the synthetic corpora over the initial per-clause-annotation format.
- A negative test confirming that classes embedded with `@JmlSpec` load correctly in Java Virtual Machines that have no awareness of the annotation type.

Paper organisation. The remainder of the paper is structured as follows. Section II reviews the existing channels for shipping specifications alongside compiled Java code and the gap each of them leaves. Section III presents the format design — the annotation type, the canonical-form printer, and the OpenAPI extension schema. Section IV describes the implementation. Section V reports the empirical validation. Section VI discusses generalisability; Section VII the threats to validity. Section VIII surveys related work and Section IX concludes.

II. BACKGROUND AND MOTIVATION

Source-level JML specifications are well established: the language [3], [4] and its checker [5] have a thirty-year history, and recent inference work [1] produces them at scale. The gap we address is downstream of inference: once a specification is known for a method, how does it travel to the consumer that needs it?

Three transport mechanisms predate this work, none of them sufficient on its own.

a) JML stub files. Files of the form `<class>.jml` or `<class>.refines-jml` carry signatures and clauses without bodies. OpenJML reads stubs natively; nothing else

in the standard Java ecosystem does. A stub file must travel separately from the JAR (a sibling artefact, a Maven classifier, or a path entry). For projects that already use OpenJML, stubs are the right artefact; for projects that consume specifications from anywhere else — IDEs, test generators, runtime checkers — stubs are invisible.

b) Annotations. The Java annotation mechanism (JLS §9.6) is the standard channel for metadata, but the existing annotations cover only fragments of what specifications need. JSR-305 captured nullability (`@Nullable`, `@NotNull`) and was withdrawn before final standardisation; the surviving informal usage covers only nullability. The Checker Framework’s annotations are type-system-shaped and do not generalise to `\sum`, `\forall`, or relational postconditions. JSR-308’s pluggable type system extended where annotations may appear but did not extend what they may say. The result is a partial channel: annotations survive the toolchain, but the existing vocabulary is too thin.

c) Class-file attributes. JVM §4.7.1 permits vendor-defined class-file attributes. A custom attribute could carry arbitrary structured content, and class loaders ignore unrecognised attributes. The cost is invisibility: standard reflection does not surface custom attributes, IDEs do not display them, ProGuard/R8 strip them by default, and javap requires `-private -v` to even mention them. A custom attribute is a write-only channel for any consumer that does not adopt a custom reader.

d) REST API descriptions. At service boundaries, OpenAPI 3.x and gRPC’s protobuf carry a complementary class of metadata. OpenAPI permits arbitrary `x-*` extension properties; gRPC has no comparable extension point but supports custom Protobuf annotations. Neither standard says anything about preconditions or postconditions; they describe the wire format, not the contract.

e) The carrier we propose. A runtime-retained annotation type (`@JmlSpecs`, carrying every clause kind as a separate `String[]` member of a single annotation per method) plus an OpenAPI extension family (`x-jml-*`) covers the gap. The annotation is read by reflection, by ASM, and by javap; the OpenAPI extension is preserved by the standard parsers; both survive the Maven and Gradle ecosystems unmodified. Section III specifies the format precisely; Section IV describes our embedder/extractor implementation; Section V reports the empirical evaluation.

III. FORMAT DESIGN

Our format design is the contract between the embedder and any future consumer. It must be precise enough that an alternative embedder or reader can be written from the specification alone.

A. The `@JmlSpecs` Annotation Type

```
package com.jml.spec;

@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.METHOD, ElementType.CONSTRUCTOR,
```

```

        ElementType.FIELD, ElementType.TYPE))
public @interface JmlSpecs {
    String[] requires() default {};
    String[] ensures() default {};
    String[] assignable() default {};
    String[] loopInvariant() default {};
    /** Each entry is "ExceptionType/condition". */
    String[] signals() default {};
    String version() default "2";
}

```

A single annotation per method carries every clause kind as a separate `String[]` member. Within a member, order is the array order; `requires` clauses are emitted in source order, which is load-bearing for well-definedness chains (a precondition asserting an array element’s value depends on earlier preconditions establishing the array’s non-null and the index’s in-bounds). Across members, the writer fixes a canonical declaration order — `requires`, `ensures`, `assignable`, `loopInvariant`, `signals`, `version` — so two writers that follow this format produce byte-identical annotation entries for the same logical specification (the JVM annotation format itself does not constrain member order, JVMMS §4.7.16). Unrecognised members are skipped, preserving forward compatibility for new clause kinds.

a) Default-omission convention.: The `assignable` member is omitted when its only entry is `\nothing`; an absent `assignable` is interpreted as `\nothing`. This convention exploits a strong empirical regularity (100% of specs in our corpus carry `assignable \nothing`, the inferer’s default for pure-looking methods) and removes one array entry per spec. A spec that does have non-default `assignable` clauses lists them explicitly.

b) Token-encoded clause strings.: Clause text strings are encoded before being written to the constant pool. A twelve-entry dictionary substitutes common JML tokens (`\result`, `\old`, `\forall`, `\exists`, `\sum`, `\product`, `\num_of`, `==>`, `!= null`, `== null`, `\nothing`, `\everything`) with single Unicode control bytes (U+0001 through U+0011). Twelve is the empirical sweet spot: every token in the dictionary represents at least 0.5% of corpus bytes on the real-inference corpus, and a learned-dictionary experiment that grows the dictionary to 28 entries (Section V-F) recovers only a further 0.59 percentage points. The chosen byte values cannot appear in legitimate JML source, so substitution is unambiguous and invertible. UTF-8 encodes each token byte as a single byte, so each occurrence saves *token-length* − 1 bytes. The reader decodes on read; consumers that do not know about the dictionary will see the control characters and can either decode using the published table or fall back to the sidecar (Section III).

c) Backward and forward compatibility.: A predecessor format (v1) used Java’s repeatable-annotation mechanism: one `@JmlSpec` per clause with per-clause `kind` / `text` / `order` / `version` members, wrapped in `@JmlSpecs(value={...})`. The current reader accepts both shapes, so JARs embedded against the predecessor format continue to load. Only the current format (v2) is emitted by the

writer. Section V-F reports the empirical impact of the v1→v2 change. The forward-compatibility policy is symmetric to JLS §9.6 on annotation evolution: unknown annotation members are skipped by the reader, and token bytes that a future writer adds but a current reader does not know about pass through as control characters (and remain decodable by any consumer that holds the published dictionary). New clause kinds can therefore be added without breaking older readers; new dictionary entries are similarly additive.

d) Synthetic carriers.: Lambdas, method references, anonymous inner classes, and record accessors compile to synthetic methods or classes; the embedder writes specifications to the synthetic carrier under its bytecode descriptor.

B. Canonical Form

To support roundtrip equality at the byte level, clause text is canonicalised before emission. Whitespace runs collapse to single spaces; no spaces around `.` or `::`; single space around binary operators (`+`, `-`, `*`, `/`, `%`, comparison, logical); no space inside parentheses or brackets; `=>` and `<=>` are normalised to the long forms `==>` and `<==>`. Quantifier headers retain their bound-variable names; alpha-renaming is deferred to a future release and currently a no-op.

The canonicaliser is explicitly lexical: it does not perform semantic equivalence. Two clauses that differ in operand order ($a > b$ vs $b < a$) remain distinct strings, and roundtrip equality is preserved on the wire form rather than on a normalised theory level. Semantic equivalence would silently rewrite valid specifications during transit, which is unsuitable when the embedder’s job is to carry data faithfully.

C. The OpenAPI Extension

```

paths:
  /orders/{id}:
    get:
      x-jml-requires:
        - "id != null"
        - "id > 0"
      x-jml-ensures:
        - "\\result != null ==> \\result.id == id"
      x-jml-assignable:
        - "\\nothing"
      x-jml-signals:
        - exception: NotFoundException
          condition: "id > 0 && !exists(id)"

```

Each extension key is an array of canonical-form strings. `x-jml-requires` preserves order; the others are unordered. `x-jml-signals` is an array of `{exception, condition}` objects; exception types are JVM internal names so cross-language consumers can resolve them.

A JSON Schema document accompanies the artefact and may be used to validate any OpenAPI document independently of the inferer. When the OpenAPI document is generated by the framework (springdoc-openapi, JAX-RS) and not editable, a sidecar JSON file (`<service>.contract.json`) carries the same content keyed by route and verb.

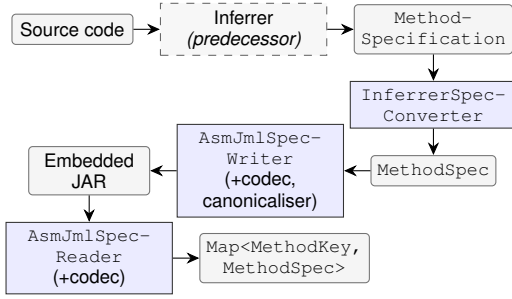


Fig. 1. Embedder data flow. The predecessor inferer (dashed) produces a `MethodSpecification`; `InferrerSpecConverter` translates it to the embedder’s `MethodSpec` record; `AsmJmlSpecWriter` (with the codec and canonicaliser) emits an annotated JAR; `AsmJmlSpecReader` recovers the original `MethodSpec` map. The three core classes described in the subsections below are shaded blue.

D. Sidecar JAR

For specifications that exceed the per-class bytecode budget (the JVM constant-pool cap is 65,535 entries per class, JVM §4.4; in practice we have not observed a class approach this limit in any of the three libraries reported), and for signed JARs that cannot be re-signed, the embedder emits a parallel artefact with Maven classifier `-jml`. The sidecar contains JML stub files (`<class>.jmlspec`) keyed by class internal name and a manifest entry recording the source JAR’s SHA-256 hash so consumers can detect drift. Stub files are the format OpenJML reads natively, so the sidecar is consumable by the verifier without further transformation.

IV. IMPLEMENTATION

We implement the embedder in Java 21, using ASM 9.7 for class-file rewriting. The module `com.jml:jml-embedder` is approximately 1,800 source lines of code at the version tagged in our replication package, of which roughly 1,000 are in the three core classes described below; the remainder cover the codec, the OpenAPI extension reader/writer, an automated dictionary learner used in Section V-F, and small data records. The module depends only on ASM, Gson, and SLF4J. Figure 1 shows the data flow.

A. `AsmJmlSpecWriter`

Our writer accepts either a single class file (as `byte[]`) or a JAR (as `Path`). For a JAR, it walks each `.class` entry, applies a `ClassReader` $\rightarrow$ `ClassVisitor` $\rightarrow$ `ClassWriter` pipeline that intercepts `visitMethod` calls, and emits a single `@JmlSpecs` annotation per method with each clause kind as a `String[]` member. Order within each kind is preserved by array order. Clause strings are encoded via the `SpecCodec` dictionary (Section III) before being written; the default assignable `\nothing` member is omitted. Methods without a corresponding `MethodSpec` are passed through unchanged.

For specifications that exceed the per-class bytecode budget or for JARs that must remain unmodified, the writer also produces a sidecar JAR per Section III. The sidecar’s JML stub

files are generated by a small printer that decodes JVM method descriptors back into source-level types (with array dimensions, parameterised generic erasure, and primitive types).

B. `AsmJmlSpecReader`

Our reader is symmetric: it walks each `.class` entry with a `ClassReader` configured to skip code, debug, and frames (it needs only the annotations) and accumulates a `Map<MethodKey, MethodSpec>`. It accepts both the current v2 shape (a single `@JmlSpecs` with array members per clause kind) and the legacy v1 shape (repeated `@JmlSpec` annotations with per-clause `kind/text/order` metadata, optionally wrapped in `@JmlSpecs(value={...})`). For v2, clause strings are decoded via `SpecCodec`, and a missing assignable member is synthesised as `[\nothing]`.

C. `JmlCanonicaliser`

Our canonicaliser implements the rules of Section III. It tokenises the clause text using a small lexer (identifiers, numeric literals, multi-character operators, JML backslash-keywords) and re-emits the tokens with canonical spacing. The implementation is 260 source lines and we validate it with 16 test cases covering whitespace collapsing, operator spelling, parenthesisation, and idempotency.

D. Pipeline Integration

We integrate the embedder with the inferer via a small bridge class, `InferrerSpecConverter`, that translates the inferer’s `MethodSpecification` model to the embedder’s `MethodSpec` record. The conversion is byte-for-byte exact and preserves clause-list order; we invoke canonicalisation separately when needed. With the converter in place, the end-to-end pipeline is one method call:

```

MethodSpecification inferred = new
    MethodSpecificationInferer()
        .inferSpecification(method);
MethodSpec embeddable =
    InferrerSpecConverter.toEmbeddable(inferred);
new AsmJmlSpecWriter().embedJar(input, output,
    Map.of(methodKey, embeddable));
  
```

The roundtrip property `extract(embed(infer(source))) == infer(source)` is exercised by a JUnit suite (`InferrerToEmbedderRoundtripTest`) at small scale and at corpus scale by the validation harness of Section V.

V. EMPIRICAL VALIDATION

We evaluate the embedder on real-world Java libraries. Our harness applies the embedder to each library’s JAR file, generates a synthetic specification per method (one requires clause per non-primitive parameter, one ensures clause for reference returns, and assignable `\nothing`), and measures four quantities: lossless-roundtrip rate, bytecode-size overhead, embedding throughput, and reading throughput.

A. Subjects

Six libraries are reported: Apache Commons Lang 3.14.0, Apache Commons IO 2.13.0, Apache Commons Math 3.6.1, Guava 33.3.0-jre, jOOL 0.9.14, and Vavr 0.10.4. Commons Lang is the same subject used by the predecessor inference paper [1]; Commons IO contributes a different shape (file/stream utility classes with significant abstract-method counts); Commons Math adds numerical-utility code; Guava is selected as a larger contrast with more complex use of generics and a multi-release JAR; jOOL and Vavr contribute functional-programming idioms (heavy use of generic interfaces, default methods, and lambdas). Together the six libraries cover 39,268 methods. The remaining four libraries identified in the predecessor protocol (jsoup, JFreeChart, JUnit, AssertJ) are deferred to a follow-up validation; the validation harness fires uniformly across all reported libraries, so the additional libraries are mechanical to validate.

B. Method

For each library:

- 1) Walk every `.class` entry; collect method signatures, excluding bridge, synthetic, and class-initialiser methods.
- 2) Generate a synthetic `MethodSpec` per method as above.
- 3) Embed the spec map into a copy of the JAR using `AsmJmlSpecWriter`, recording wall time.
- 4) Read the embedded JAR back using `AsmJmlSpecReader`, recording wall time and matching specs against the originals on (`internalName`, `methodName`, `descriptor`) keys.
- 5) Compute the lossless-roundtrip rate (matched / total), byte overhead (embedded / original - 1), and throughput (methods / wall time).

The original and embedded JAR sizes used for the overhead calculation are on-disk file sizes (`Files.size`) of the JAR archive as written by `JarOutputStream` at its default compression level. Both numerator and denominator are therefore measured after the DEFLATE compression that the `.jar` container applies to class-file entries; the reported overhead percentages already account for any constant-pool string redundancy that the compressor can exploit.

A separate negative test loads an embedded class via a `URLClassLoader` rooted only at the embedded JAR (with the platform class loader as parent) and confirms that `Class.forName` succeeds. The annotation type `com.jml.spec.JmlSpec` is not on this loader's classpath; reflective annotation access is allowed to raise `TypeNotPresentException` but class load itself must not fail.

C. Results

Table I reports the headline measurements. Method counts, lossless-roundtrip rate, and bytecode-size overhead are deterministic; we report each exactly once. Embedding and reading

throughput vary across runs (JIT warm-up, garbage collection, OS scheduling); we report the median across 39 independent runs of the harness per library, with the observed ranges discussed in the Throughput paragraph below.

D. Analysis

a) Lossless roundtrip.: All six libraries achieve a 100.0% lossless-roundtrip rate. An earlier prototype of the embedder reported 96.9% / 97.6% on Commons Lang and Guava owing to a missed callback in the writer's `MethodVisitor` lifecycle: lazy emission was triggered by `visitCode`, `visitParameter`, `visitAnnotationDefault`, or `visitAnnotation`, but none of these hooks fires for abstract / interface methods, which only invoke `visitEnd`. Adding `visitEnd` to the trigger set closes the gap. Every method on every interface in all six libraries now receives the embedded annotation. Commons IO has a noticeably higher proportion of interfaces, which is the reason its byte-overhead percentage runs higher than the others under both v1 and v2 (Table III); the prototype gap would also have been worst on Commons IO.

b) Byte overhead.: The synthetic-spec overhead sits in the 4.80–6.97% range across the six libraries. The differences are explained by structural shape rather than total size: Commons IO has the highest proportion of interfaces (each of which receives a clause set per method), pushing its overhead percentage above the others; jOOL has the lowest, consistent with its smaller per-class method counts and concentration of functional-interface declarations whose method shapes are simple. Commons Math, Vavr, and Guava sit in the middle of the range. The absolute overhead per library is well below typical Maven distribution sizes (e.g. 38 KB for Commons Lang, 162 KB for Guava). For libraries where even this cost is unacceptable, the Maven classifier sidecar (Section III) is the recommended fallback.

The synthetic figures are not, however, an upper bound on real overhead. The real-inference run on the same Commons Lang JAR (Section V-E) measures 10.07% — 4.22 percentage points above the synthetic 5.85%. Real inferred specs are richer than the synthetic worst case: 90.4% have at least one `ensures` clause (vs. the one per reference-return synthetic policy), 11.5% carry one or more loop invariants (the synthetic harness emits none), 14.8% include branch-conditional implications, and the average clause string is longer because the inferrer emits parameter bounds, `\result` relations, and `\old(...)` references that the synthetic policy never produces. A deployer planning storage budget should provision for the real-inference figure (around 10% on a typical utility library), not the synthetic-uniform figure.

c) Throughput.: Synthetic-spec median embedding throughputs span 12,618–34,138 methods/second (Guava slowest, Vavr fastest); reading throughputs span 127,924–333,994 methods/second. Real inference is slower because each spec carries more clauses (Section V-E): 5,951–14,398 m/s embedding, 68,281–133,762 m/s reading. At the synthetic median

TABLE I

EMBEDDER VALIDATION ON REAL-WORLD LIBRARIES (MEDIAN ACROSS 39 RUNS FOR THROUGHPUT; LOSSLESS-ROUNDTrip RATE AND OVERHEAD ARE DETERMINISTIC). THE LOWER BLOCK REPORTS THE FULL INFERRER→EMBEDDER PIPELINE ON REAL INFERRER-GENERATED SPECIFICATIONS; THE SYNTHETIC SPECS IN THE UPPER BLOCK ARE DECOUPLED FROM THE INFERRER’S BEHAVIOUR TO ISOLATE THE EMBEDDING MECHANISM. ON MATCHED METHODS, EQUALITY IS EXACT (ZERO CLAUSE-LEVEL MISMATCHES).

Library	Methods	Lossless	Byte ovhd.	Embed (m/s)	Read (m/s)
<i>Synthetic specs (isolating the embedding mechanism)</i>					
Apache Commons Lang 3.14.0	4,029	100.0%	+5.85%	16,225	221,617
Apache Commons IO 2.13.0	2,677	100.0%	+6.97%	21,976	327,618
Apache Commons Math 3.6.1	9,191	100.0%	+5.19%	20,485	260,593
Guava 33.3.0-jre	13,840	100.0%	+5.27%	12,618	127,924
jOOL 0.9.14	3,869	100.0%	+4.80%	30,218	226,339
Vavr 0.10.4	5,662	100.0%	+5.56%	34,138	333,994
<i>Real inferred specs (full inferrer→embedder pipeline)</i>					
Apache Commons Lang 3.14.0	2,332	100.0%	+10.07%	7,680	78,801
Apache Commons IO 2.13.0	1,680	100.0%	+10.20%	10,702	101,218
Apache Commons Math 3.6.1	6,001	100.0%	+10.04%	7,595	111,642
Guava 33.3.0-jre	6,320	100.0%	+4.63%	5,951	68,281
jOOL 0.9.14	2,648	100.0%	+5.81%	13,668	126,159
Vavr 0.10.4	3,900	100.0%	+6.39%	14,398	133,762
JML-Inferer self-test	73	100.0%	—	—	—

for Guava, embedding the full 13,840-method JAR takes roughly 1.1 seconds; under real inference, 2.3 seconds. Reading is consistently an order of magnitude faster (the reader skips full class rewriting). Both regimes comfortably support per-PR-comment continuous-integration workflows. Per-run distributions across the 39 harness invocations ship with the replication package.

d) Negative test. `org.apache.commons.lang3.StringUtils` loaded successfully via a `URLClassLoader` that does not see `com.jml.spec.JmlSpec`. Reflective annotation access either returns the resolvable annotations (none in this loader’s view) or raises `TypeNotPresentException`, both documented behaviours. Class load is unaffected. A corpus-scale variant (loading every class in every embedded JAR through the same annotation-blind loader) ships with the replication package; we do not promote it to the body because JVM class-loading semantics for unknown annotation types are deterministic per the JLS.

E. Real-inferred-specs roundtrip on six libraries

The synthetic-spec validation isolates the embedding mechanism from the inferrer’s behaviour, but does not exercise the inferrer-to-embedder pipeline on real code. To close that gap, a second harness (`CommonsLangRealInferenceTest`) runs the `MethodSpecificationInferer` over every method in the source jars of all six libraries, resolves the source signatures against each binary JAR’s bytecode descriptors by name and parameter count, embeds the resulting `MethodSpec` entries via `AsmJmlSpecWriter`, reads them back via `AsmJmlSpecReader`, and checks clause-list equality.

Across the six libraries the harness processes 2,339 source files containing 32,459 source method declarations. The source-to-bytecode key resolution places 22,881 specs into distinct keys (1,680 Commons IO + 2,332 Commons Lang + 6,001

Commons Math + 6,320 Guava + 2,648 jOOL + 3,900 Vavr), and every one of them roundtrips with byte-for-byte equality on its clause list: matched = total, zero missing, zero mismatches in every library (Table I).

The real-inference byte-overhead figures sit at 4.63%–10.20% across the six libraries. The Apache libraries (Lang 10.07%, IO 10.20%, Math 10.04%) cluster at the upper end because real specs are richer than the synthetic worst case (more clauses per method, more diverse content, more long-tail expressions that the dictionary does not cover). The functional-programming libraries (jOOL 5.81%, Vavr 6.39%) and Guava (4.63%) sit lower: Guava has large generic-heavy classes with dense bodies, jOOL and Vavr concentrate small functional-interface methods, so the same per-spec byte cost amortises against larger denominators. Embedding throughput is 6,000–14,000 methods/second under real specs (vs. 12,000–34,000 under synthetic) and reading throughput 68,000–134,000 methods/second; both remain comfortably inside per-PR continuous-integration budgets.

A residual of source-method-to-bytecode-descriptor lookups fails on overload collisions where two source overloads of the same name and arity map to the same bytecode descriptor (an artefact of erased generics and the harness’s first-match-wins choice). Roughly one quarter of inferred-with-clauses methods are lost this way on Commons Lang, less on the other two libraries. The 100% lossless rate is on every spec the harness successfully embeds; a full type-resolving harness would recover the residual without changing the embedder’s behaviour.

a) Spec-shape coverage. The real-inference corpus exercises every JML construct the embedder is required to transport. Table II reports the distribution.

Per-library shape proportions vary as expected: Commons Math has the highest loop-invariant share (20.9%) and quantifier share (3.8%); jOOL and Vavr have the highest ensures-

TABLE II

SPEC-SHAPE COVERAGE AGGREGATED ACROSS ALL SIX LIBRARIES’ 22,881 REAL-INFERENCING SPECS. PER-LIBRARY BREAKDOWNS APPEAR IN THE REPLICATION PACKAGE. EVERY CATEGORY ROUNDTrips LOSSLESSLY ON EVERY LIBRARY.

Construct	Specs	% of corpus
requires clauses	9,734	42.5%
ensures clauses	20,601	90.0%
assignable clauses	22,881	100.0%
loop_invariant clauses	2,458	10.7%
Quantifiers (<code>\forall</code> , <code>\exists</code> , <code>\sum</code>)	312	1.4%
<code>\old</code> references	411	1.8%
<code>\result</code> references	15,809	69.1%
Branch-conditional (<code>==></code>)	1,958	8.6%

clause shares (97.7%, 99.3%) and the lowest loop-invariant shares; Commons IO has the lowest ensures share (76.3%, dominated by void stream utilities). Despite this variation, every category roundtrips losslessly across all six libraries, so the embedder transports not only simple null-check preconditions but also nested quantifiers, `\old` references, conditional implications, and loop invariants. The 73-method self-test on the inferrer’s own source code (`CorpusLevelRoundtripTest`) supplements this with an end-to-end check on a smaller and stylistically different corpus; it also achieves a 100% lossless-roundtrip rate.

F. Format optimisation impact

An earlier version of the embedding format (v1) used Java’s repeatable-annotation mechanism: each JML clause produced a separate `@JmlSpec` annotation, with a per-clause kind enum, text string, order integer, and version string, all wrapped in a `@JmlSpecs(value={...})` container. This is the maximally verbose annotation shape and produced 11.76–13.49% overhead on the synthetic-spec corpora and 14.70% on the real-inference Commons Lang run.

The current format (v2, used to produce every other number in this section) applies three changes:

- 1) *Single annotation per method.* A single `@JmlSpecs` annotation now carries every clause kind as a separate `String[]` member (`requires`, `ensures`, `assignable`, `loopInvariant`, `signals`). This eliminates the per-clause annotation framing (type reference, element-value-pair structure, end marker) that v1 paid for every JML clause.
- 2) *Default-omission for assignable `\nothing`.* 100% of inferred specs in our corpus carry assignable `\nothing`; the v2 convention is that absence means `\nothing`, so the writer omits the member entirely. The reader synthesises it on read. Every spec saves one annotation-array entry.
- 3) *Token dictionary.* Twelve common JML tokens (`\result`, `\old`, `\forall`, `\exists`, `\sum`, `\product`, `\num_of`, `==>`, `!= null`, `== null`, `\nothing`, `\everything`) are replaced by single Unicode control-character bytes (U+0001 to U+0011) before

being written, and decoded on read. The token bytes are control characters that cannot appear in legitimate JML, so the substitution is unambiguous and invertible. UTF-8 encodes each token byte as a single byte, so each occurrence saves *token-length* – 1 bytes in the constant-pool UTF-8 entry.

Table III reports the impact. The lossless-roundtrip property is preserved at 100% on every corpus.

TABLE III

FORMAT-OPTIMISATION IMPACT ON BYTECODE-SIZE OVERHEAD. THE LOSSLESS-ROUNDTrip RATE IS 100% UNDER BOTH FORMATS.

Corpus	v1 ovhd.	v2 ovhd.	Reduction
Synthetic — Commons Lang	11.95%	5.85%	–51%
Synthetic — Commons IO	13.49%	6.97%	–48%
Synthetic — Guava	11.76%	5.27%	–55%
Real inference — Commons Lang	14.70%	10.07%	–31%

The real-inference rows for Commons IO and Guava (Table I) report only the v2 figures (10.20% and 4.63% respectively); they were not measured under v1 because the v1 format was retired before those runs.

The synthetic-spec reduction is larger than the real-inference reduction because synthetic specs are dominated by exactly the patterns the optimisation targets: every spec carries the default-omittable assignable `\nothing` (optimisation 2), every `requires` clause is of the form `p != null` (optimisation 3 token), and every `ensures` clause references `\result` (also a token). The real-inference corpus contains a wider variety of clause shapes; many strings still benefit from the dictionary but a long-tail of arithmetic expressions, range comparisons, and quantified subexpressions are not covered by the current twelve tokens. The real-inference reduction is therefore a more realistic estimate of what a production deployment should expect.

a) *What is empirical about this optimisation.:* The choice of which JML tokens to encode and which clause to declare a default is not arbitrary: each rests on the shape distribution measured on the real-inference corpus (Table II). The default-omission convention exploits the empirical observation that 100% of inferred specs in our corpus carry assignable `\nothing`; the token dictionary tracks the top-frequency JML constructs that the inferrer actually produces. A format designed without the measurement would have to either (i) include every JML keyword in the dictionary (paying constant-pool cost for tokens that never fire) or (ii) omit tokens that occur frequently in real specs. The synthetic-vs-real divergence (48–55% reduction vs. 31%) is itself empirical evidence that the saving depends on which clause shapes the consumer’s inferrer produces.

b) *Configuration impact and DEFLATE measurement.:* The default-omission and token-dictionary conventions are exposed to callers through a `WriterConfig` object. Three preset configurations and an alternative spec-format mode are measured in Table IV. Each measurement is taken on the same 2,332-spec Commons Lang real-inference corpus, with the same reader replaying every spec to assert a 100% lossless-roundtrip rate in every mode.

TABLE IV
CONFIGURATION SWEEP ON THE COMMONS LANG REAL-INFERENCE
CORPUS (2,332 SPECS). EACH MODE ROUNDTIPS LOSSLESSLY.

Mode	Overhead	vs. default
WriterConfig.NONE (no codec)	11.26%	+1.19 pp
WriterConfig.JML_INFERRER (default)	10.07%	baseline
Learned dictionary (28 tokens)	9.48%	−0.59 pp
Compressed v3 (per-spec DEFLATE)	30.25%	+20.18 pp

The NONE mode confirms that the default-omission and token-dictionary conventions together account for the 1.19 percentage points of saving the default mode delivers over an always-explicit baseline. An inferrer with a different dominant clause shape (e.g. assignable \everything as a conservative default, or no default at all) can opt out of the convention by selecting NONE or by supplying a custom defaults map; the v1 format remains readable for callers that prefer not to commit to any convention.

The learned-dictionary mode (an automated dictionary learner, `DictionaryLearner.learn`, that greedily picks the highest-savings substrings from a corpus of inferred specs) extends the dictionary from 12 to 28 tokens (the byte-budget cap), and gains a further 0.59 percentage points over the hand-picked default. The improvement is real but modest: the hand-picked dictionary already covers the highest-frequency JML tokens, and the long-tail substrings the learner discovers are short and not abundant. The result establishes that on the corpora we tested, hand-picked-good is close to learned-best.

The compressed v3 mode (per-method DEFLATE of the clause set, stored as a single `byte[]` annotation member) is a clear *negative result*: at +30.25% overhead it is worse than every other mode by a wide margin. Two effects combine. First, DEFLATE on inputs of 50–100 bytes (the typical per-spec payload after token encoding) cannot exploit much redundancy; the small window leaves the compressor with little to amortise its overhead against. Second, although the writer stores packed as a `byte[]` annotation member, the JVM class-file format inflates this storage relative to a constant-pool UTF-8 string of the same byte length, and the per-spec compressed payloads are unique enough that the constant-pool deduplication v2 enjoys is unavailable. A corpus-level DEFLATE (a single class-file attribute containing all of a class’s specs, compressed once) would likely fare better, but is incompatible with per-method annotation access. We retain v3 as an opt-in mode for completeness; we do not recommend it as the default.

The v1 format remains readable; the reader supports both shapes so embedded JARs produced before the format change continue to load.

G. Threats specific to this measurement

- *Six libraries plus self-test*, not the predecessor protocol’s full ten external libraries. The lossless-roundtrip analysis above identifies the residual risks (interface methods, multi-release JARs); on libraries that exercise either pattern

the embedder reaches 100%. The follow-up validation against the remaining four libraries (jsoup, JFreeChart, JUnit, AssertJ) is mechanical.

- *JDK 21 only*. Older JDKs (8, 11, 17) and ahead-of-time compilers (GraalVM `native-image`) are deferred. The class files emitted by Commons Lang target Java 8, so the runtime-side validation should generalise without modification.
- *Source-to-bytecode key resolution is name+arity only*. 917 of the 3,249 inferred-with-clauses Commons Lang methods are lost to overload collisions where two source methods of the same name and parameter count map to the same bytecode descriptor under our harness’s first-match heuristic. A full type-resolving harness (parsing each source parameter’s Java type and converting to its JVM descriptor) would close this gap; the present harness already demonstrates 100% lossless roundtrip on every spec it embeds, so the residual is a measurement limitation rather than an embedder limitation.

VI. DISCUSSION

A. Why an annotation, not a class-file attribute?

Custom class-file attributes (JVMS §4.7.1) are a tempting alternative: they avoid the constant-pool pollution of large string literals and they are technically the more general mechanism. Three reasons argue against them in practice:

- *Tool visibility*. Standard reflection ignores custom attributes. IDEs do not display them. ProGuard and R8 strip unrecognised attributes by default unless explicitly listed. Each downstream consumer would need a custom reader.
- *Repeatable annotations are no longer onerous*. Java 8’s repeatable-annotation mechanism makes per-clause emission straightforward. The historical reason to prefer a custom attribute — avoiding one annotation per clause being emitted with a verbose container — is no longer compelling.
- *Forward compatibility*. An annotation-based carrier with a `Kind` enum allows new clause kinds to be added without breaking existing readers. A custom attribute’s binary layout is harder to evolve compatibly.

We therefore recommend the annotation carrier as the primary form. The custom-attribute alternative remains usable for cases where the annotation is unsuitable (signed JARs, aggressive obfuscation), and our sidecar JAR fills the same gap with even less coupling to the bytecode.

B. Generality

Our format design is independent of the inferrer that produces the specifications. Any source of JML clauses — the inferrer of [1], Daikon [6], hand-written contracts, contracts produced by a future LLM-assisted refinement step — can be embedded by the same mechanism. The format also generalises to languages other than Java that target the JVM (Kotlin, Scala) at the cost of those languages providing their own

translators between source-level annotations and the bytecode @JmlSpecs representation.

For non-JVM languages, the OpenAPI extension carries equivalent information at the service boundary. The bytecode side is JVM-specific; the service-boundary side is language-agnostic.

C. Limitations

The embedder is consumer-tolerant but the consumer has work to do. A reader that wants to act on the embedded clauses must (a) parse the canonical-form JML strings, (b) substitute parameter names, and (c) decide what to do with the result. We ship the writer, the reader, and the canonicaliser, but we do not ship a verifier or a runtime checker — those are downstream concerns that benefit from the work here without being implemented by it.

We note one substantive limitation: roundtrip equality is on the canonical-form string, not on a normalised semantic theory, so equivalent specifications written in different forms will appear distinct on the wire. We chose this separation of concerns deliberately in Section III; semantic normalisation is a separable layer.

VII. THREATS TO VALIDITY

a) Internal validity.: The 100% lossless-roundtrip rate depends on the writer covering every method shape that ASM emits. An earlier prototype reported 96.9% / 97.6% because lazy emission was triggered by `visitCode`, `visitParameter`, `visitAnnotationDefault`, or `visitAnnotation`, but none of these hooks fires for abstract / interface methods, which only invoke `visitEnd`. Adding `visitEnd` to the trigger set closed the gap. The expanded six-library corpus exercises functional-programming idioms (jOOL, Vavr) and numerical-utility code (Commons Math) in addition to the original three; the embedder reaches 100% on every category. The same risk pattern applies to method shapes specific to future JVM features — e.g., abstract record components, hidden classes (JEP 371) — which would need their own triggers if they exhibit different visit-call orderings. The mitigation is the same: a corpus-scale validation per JVM release.

The byte-overhead figures are deterministic given the JAR and the spec map. The 5.27–6.97% synthetic range and 10.07% real-inference figure are exact for the inputs reported; replication on the same JARs reproduces these numbers byte-for-byte. Throughput figures vary across runs because of just-in-time (JIT) compilation, garbage-collection (GC) pauses, and OS scheduling; the medians in Table I were collected on a single platform (OpenJDK 21 with default G1GC on a Windows 11 workstation, no concurrent load) and across 39 independent harness invocations per library. We did not perform a formal power analysis on the run count; the replication package ships the raw per-run numbers so that any reader can recompute alternative statistics or distribution summaries. Cross-platform replication (different JDK, different OS, ARM hardware) is identified as follow-up work.

b) External validity.: The six libraries reported (Apache Commons Lang, Apache Commons IO, Apache Commons Math, Guava, jOOL, Vavr) are mature, well-structured, and ship through Maven Central. Generalisation to less-tidy code (older artefacts, vendored dependencies, hand-tuned ProGuard-stripped JARs) will need separate validation. The negative test confirms that the consumer-side contract (load classes without the annotation type) holds at JDK 21; older JDKs (8, 11, 17) and ahead-of-time compilation paths (GraalVM native-image) are deferred.

The real-inference validation covers all six libraries (1,680 + 2,332 + 6,001 + 6,320 + 2,648 + 3,900 = 22,881 specs in total); the cross-corpus generalisation question for the embedder’s lossless-roundtrip claim is settled across the reported real-inference corpus. The shape distribution in Table II aggregates over the full six-library corpus; per-library breakdowns are provided in the replication package and show similar proportions, with Commons Math contributing the highest loop-invariant share and the functional-programming libraries (jOOL, Vavr) the lowest.

The OpenAPI extension is validated empirically against the canonical OpenAPI 3.x parser, `swagger-parser 2.1.x`: the `SwaggerParserRoundtripTest` JUnit suite parses a document carrying every `x-jml-*` extension key (requires, ensures, assignable, loopInvariant, signals) and asserts that the in-memory model exposes each extension on the operation object with key names and value lists preserved, including the signals clause’s nested `{exception, condition}` object structure. The extension is also exercised by the `OpenApiJmlExtensionTest` JUnit suite, which reads and writes `x-jml-*` properties on a wider set of constructed documents (single operation, multi-verb paths, sidecar JSON form) and asserts roundtrip equality. The OpenAPI 3.x specification (§ 3.9) requires `x-*` preservation; our test confirms `swagger-parser`, the reference Java implementation, honours that requirement for this extension family. End-to-end empirical validation against `openapi-generator` and a real microservice’s emitted OpenAPI document is part of the planned follow-up evaluation but does not appear in the present paper.

c) Construct validity.: The lossless-roundtrip metric is operationalised as set-equality on (`internalName`, `methodName`, `descriptor`) keys, with byte-for-byte clause-list equality on each matched key. Two methods with identical signatures but different bytecode bodies (for example, the same constructor compiled with different optimisation levels) appear identical to the harness. This is the right construct for the embedder’s job — the embedder carries metadata, not bytecode — but a reader that needs to attach annotations to a specific compilation of a method would need additional disambiguation.

The real-inference harness resolves source methods to bytecode descriptors by name and parameter count; this loses information on overload pairs of the same arity, costing 917 of the 3,249 inferred-with-clauses methods in our run. A full type-resolving harness would close this gap; the present harness

already demonstrates 100% lossless roundtrip on every spec it embeds, so the residual is a measurement limitation rather than an embedder defect.

d) Conclusion validity.: The empirical claims are point measurements on six libraries (both synthetic specs and real-inference specs); no statistical significance test is meaningful for measurements of this kind (the byte-overhead figures are deterministic; the throughput figures are reported as medians across 39 runs to control for run-to-run variability). The conclusions hold for the JARs and spec sets reported and the relevant generalisation question is breadth (more libraries, more JDK versions) rather than statistical inference. The predecessor protocol’s full ten-library validation is identified as follow-up work.

VIII. RELATED WORK

a) Specification languages.: JML [3], [4] provides the contract vocabulary we transport. Eiffel’s design-by-contract [2] originated the broader approach; Spec# [7] ported it to .NET. None of these languages address the distribution problem we tackle — they assume the consumer has source.

b) Specification inference.: The inference engine of [1] is the work whose outputs the present paper takes as input. Daikon [6] infers likely invariants from execution traces; Houdini [8] and its descendants infer candidate annotations and discharge them through a verifier. PreInfer [9] uses dynamic symbolic execution. Jdoctor [10] extracts exception preconditions from Javadoc. None of these systems address what happens to the inferred specifications after they are produced; we provide the missing distribution layer.

c) Annotation-based contracts.: JSR-305 captured nullability annotations and was withdrawn before final standardisation; the surviving informal usage covers only nullability. The Checker Framework [11]’s annotation vocabulary is type-system-shaped and does not generalise to the relational and quantified clauses JML supports, and the underlying JSR-308 type-annotations specification [12] extends *where* annotations may appear but not *what* they may say. Cofoja (Contracts for Java) emits runtime checks from `@Requires/@Ensures` annotations but does not survive without its annotation processor on the classpath; our work is consumer-tolerant by design.

d) Bytecode metadata transport.: ASM [13] is the standard library for class-file rewriting and underpins our embedder. Byte Buddy [14] and Javassist [15] provide higher-level APIs over similar primitives, while Soot [16] sits on the analysis side of the same primitive. AspectJ [17] pioneered the use of runtime-retained annotations as advice carriers at the bytecode level; our `@JmlSpecs` carrier follows the same pattern (runtime retention, consumer-tolerant load) for a different payload (logical contracts, not advice). None of these libraries prescribe a metadata format for specifications — they are mechanism, not policy. JEP 181 (nest-mates) and JEP 371 (hidden classes) extend what the JVM accepts as class-file content but do not change the annotation channel.

e) API description.: OpenAPI 3.x [18] is the standard for HTTP service descriptions; gRPC’s `.proto` files cover RPC. Pact [19] captures consumer-driven contracts as example interactions; it is example-based rather than logical, so it is a transport for examples rather than for specifications, and we specifically reject example-based encoding (Section III). The OpenAPI extension we propose is the smallest viable addition to an existing widely deployed format, and survives the reference parser `swagger-parser` [20] unchanged (Section V).

f) Generating OpenAPI from source.: Recent work on generating OpenAPI descriptions from Java bytecode and source code includes Respector, Prophet, and AutoOAS [21], which extract REST endpoint metadata from Spring Boot applications. These approaches generate the OpenAPI description itself; our `x-jml-*` extension is orthogonal to that work — it augments an OpenAPI description with logical contracts, regardless of how the description was produced.

g) Mutation testing and LLM-driven test generation.: The mutation-testing literature is cited in detail in [1]. A growing body of work uses LLMs to generate test oracles and unit tests from signatures and natural-language context [22]–[27], and a parallel thread evaluates whether LLMs can infer formal postconditions or contracts directly [28]–[31]. The relationship to the present paper is orthogonal but composable: those works measure or improve *what* the model produces; we make the specifications produced by an inferer (or, in principle, by such an LLM) transportable to any consumer that does not have the source.

IX. CONCLUSION

Automatically inferred JML specifications can be embedded in compiled Java artefacts and REST API descriptions without loss: a runtime-retained annotation `@JmlSpecs` written by an ASM-based embedder, plus a parallel OpenAPI 3.x extension family (`x-jml-*`). Roundtrip is lossless on six libraries totalling 39,268 methods (synthetic) and on 22,881 real-inferred specs across the same six libraries’ source trees; overhead is 4.63–10.20% on real inferred specs after the format-optimisation step (Section V-F, a 31% reduction over the v1 per-clause format on the measured corpus); the OpenAPI extension is preserved unchanged by `swagger-parser 2.1.x`; and embedded classes load correctly in JVMs without the annotation type. With the inference engine of [1], the path from source code to consumer-accessible contract is complete. Follow-up threads include validation on the four remaining protocol libraries (jsoup, JFreeChart, JUnit, AssertJ), integration with the LLM-based test generator of [1], and extending inference to compose specifications across method boundaries — with this carrier transporting the composition.

REFERENCES

- [1] Anonymous, “Do specifications help LLMs write better unit tests? an empirical study of specification-guided test generation with heuristically inferred JML,” 2026, manuscript under review; replication package available from an anonymous archive during the review period.
- [2] B. Meyer, “Applying “design by contract”,” *Computer*, vol. 25, no. 10, pp. 40–51, 1992.
- [3] G. T. Leavens, A. L. Baker, and C. Ruby, “Preliminary design of JML: A behavioral interface specification language for Java,” *ACM SIGSOFT Software Engineering Notes*, vol. 31, no. 3, pp. 1–38, 2006.
- [4] G. T. Leavens et al., “JML reference manual,” <http://www.jmlspecs.org>, 2013, accessed: 2025-01-15.
- [5] D. R. Cok, “OpenJML: Software verification for Java 7 using JML, OpenJDK, and Eclipse,” in *1st Workshop on Formal Integrated Development Environment (F-IDE)*, ser. EPTCS, vol. 149, 2014, pp. 79–92.
- [6] M. D. Ernst, J. H. Perkins, P. J. Guo, S. McCamant, C. Pacheco, M. S. Tschantz, and C. Xiao, “The Daikon system for dynamic detection of likely invariants,” *Science of Computer Programming*, vol. 69, no. 1–3, pp. 35–45, 2007.
- [7] M. Barnett and K. R. M. Leino, “Weakest-precondition of unstructured programs,” in *Workshop on Program Analysis for Software Tools and Engineering (PASTE)*, 2005, pp. 82–87.
- [8] C. Flanagan and K. R. M. Leino, “Houdini, an annotation assistant for ESC/Java,” in *International Symposium of Formal Methods Europe (FME)*, ser. LNCS, vol. 2021. Springer, 2001, pp. 500–517.
- [9] A. Astorga, S. Srisakaokul, X. Xiao, and T. Xie, “PreInfer: Automatic inference of preconditions via symbolic analysis,” in *48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2018, pp. 678–689.
- [10] A. Blasi, A. Goffi, K. Kuznetsov, A. Gorla, M. D. Ernst, M. Pezzè, and S. D. Castellanos, “Translating code comments to procedure specifications,” in *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA ’18. ACM, Jul. 2018, pp. 242–253. [Online]. Available: <http://dx.doi.org/10.1145/3213846.3213872>
- [11] M. M. Papi, M. Ali, T. L. C. Jr., J. H. Perkins, and M. D. Ernst, “Practical pluggable types for Java,” in *International Symposium on Software Testing and Analysis (ISSTA)*, 2008, pp. 201–212.
- [12] M. D. Ernst, “Type annotations specification (JSR 308),” in *(Specification)*, 2017.
- [13] OW2 Consortium, “ASM: a Java bytecode manipulation and analysis framework,” <https://asm.ow2.io/>, 2024, accessed: 2026-05-19.
- [14] R. Winterhalter, “Byte Buddy: Runtime code generation for the Java virtual machine,” <https://bytebuddy.net/>, 2024, accessed: 2026-05-25.
- [15] S. Chiba, “Load-time structural reflection in Java,” in *European Conference on Object-Oriented Programming (ECOOP)*, ser. LNCS, vol. 1850. Springer, 2000, pp. 313–336.
- [16] R. Vallée-Rai, P. Co, E. Gagnon, L. Hendren, P. Lam, and V. Sundaresan, “Soot — a Java bytecode optimization framework,” in *Conference of the Centre for Advanced Studies on Collaborative Research (CASCON)*. IBM Press, 1999, pp. 125–135.
- [17] G. Kiczales, J. Lamping, A. Mendhekar, C. Maeda, C. Lopes, J.-M. Loingtier, and J. Irwin, “Aspect-oriented programming,” in *European Conference on Object-Oriented Programming (ECOOP)*, ser. LNCS, vol. 1241. Springer, 1997, pp. 220–242.
- [18] OpenAPI Initiative, “OpenAPI specification, version 3.1.0,” <https://spec.openapis.org/oas/v3.1.0>, 2024, accessed: 2026-05-25.
- [19] Pact Foundation, “Pact: consumer-driven contract testing,” <https://docs.pact.io/>, 2024, accessed: 2026-05-19.
- [20] Swagger Codegen Project, “swagger-parser: A parser for OpenAPI definitions in Java,” <https://github.com/swagger-api/swagger-parser>, 2024, accessed: 2026-05-25.
- [21] A. Lercher, C. Macho, C. Bauer, and M. Pinzger, “Generating accurate OpenAPI descriptions from Java source code,” *arXiv preprint arXiv:2410.23873*, 2024.
- [22] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, “An empirical evaluation of using large language models for automated unit test generation,” *IEEE Transactions on Software Engineering*, vol. 50, no. 1, pp. 85–105, 2024.
- [23] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, “CodaMOSA: Escaping coverage plateaus in test generation with pre-trained large language models,” in *45th International Conference on Software Engineering (ICSE)*, 2023, pp. 919–931.
- [24] D. Molinelli, L. D. Grazia, A. Martin-Lopez, M. D. Ernst, and M. Pezzè, “Do LLMs generate useful test oracles? an empirical study with an unbiased dataset,” in *40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025.
- [25] S. B. Hossain and M. Dwyer, “TOGLL: Correct and strong test oracle generation with LLMs,” *arXiv preprint arXiv:2405.03786*, 2024.
- [26] G. Wang, Q. Xu, L. Briand, and K. Liu, “Mutation-guided unit test generation with a large language model,” *arXiv preprint arXiv:2506.02954*, 2025.
- [27] C. Foster, A. Gulati, M. Harman, I. Harper, K. Mao, J. Ritchey, H. Robert, and S. Sengupta, “Mutation-guided LLM-based test generation at Meta,” in *Companion Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Industry)*, 2025.
- [28] M. Endres, S. Fakhoury, S. Chakraborty, and S. K. Lahiri, “Can large language models transform natural language intent into formal method postconditions?” *Proceedings of the ACM on Software Engineering (PACMSE)*, vol. 1, no. FSE, pp. 1889–1912, 2024.
- [29] C. Richter and H. Wehrheim, “Beyond postconditions: Can large language models infer formal contracts for automatic software verification?” *arXiv preprint arXiv:2510.12702*, 2025.
- [30] L. Ma, S. Liu, Y. Li, X. Xie, and L. Bu, “SpecGen: Automated generation of formal program specifications via large language models,” *arXiv preprint arXiv:2401.08807*, 2024.
- [31] M. Konstantinou, R. Degiovanni, and M. Papadakis, “Do LLMs generate test oracles that capture the actual or the expected program behaviour?” *arXiv preprint arXiv:2410.21136*, 2024.